

Working with Salesforce Data in Lightning Web Components

Jul 03, 2025 | Sheetal Chougule



Agenda

salesforce

01

What is LDS, Benefits

02

Base Lightning Components with Lightning Data Service

03

Wire service with Lightning Data Service & UI API

04

Wire Service with Apex Methods

05

Imperative Apex Calls

06

Configuring LWC in App Builder

Lightning Data Service

Standard Controller of Lightning Components

- load, create, edit, or delete a record in component without requiring Apex code. Lightning Data Service handles sharing rules and field-level security for you.
- Use Lightning Data Service to read and modify Salesforce data in your components.
- Built on highly efficient local storage that's shared across all components that use it. Records loaded in Lightning Data Service are cached and shared across components.
- **Lightning Data Services consist of following:**
 - `force:recordPreview` tag
 - `getNewRecord()` method
 - `saveRecord()` method
 - `deleteRecord()` method
 - `recordUpdated` Event
 - `changeType` event parameter to check record update status



What LDS offers?

03. CRUD

Does not require explicit CRUD check

02. Caching

proactive cache population, shared instance of the record data across components

01.No SOQL

Avoid multiple queries to fetch data. Improves performance by saving time required to fetch data from server.



04. No Apex

Prebuilt controller methods avoid need for DMLs

05. Consistency

Client-side data storage makes all component sharing data consistently.

06. Notifications

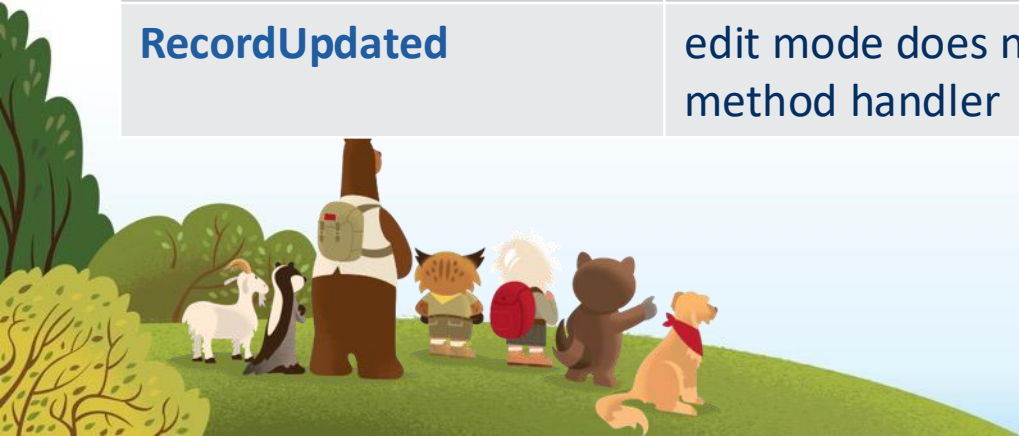
Data and record changes notify other components and refresh components.

07. Offline access for Salesforce1

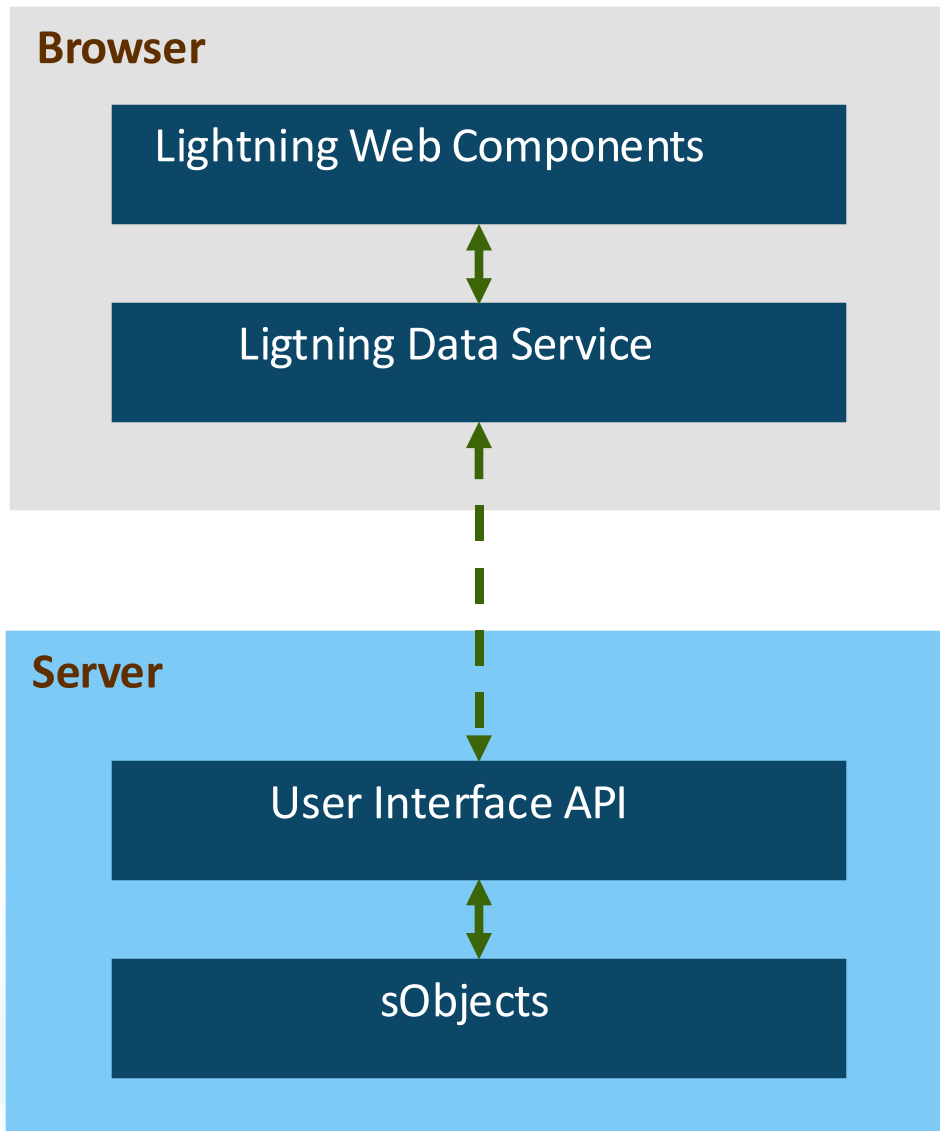
Key Attributes of LDS

use force:recordData Lightning component to declare Lightning Data Service

Attribute	Purpose
RecordId	represents record to be loaded, deleted or updated. If its null then it indicates record to be inserted
LayoutType	Layout from which field needs to be loaded. Currently it supports Full or Compact
Fields	If Layout does not contains field, we can use this attribute. Either one of LayoutType or Fields is mandatory.
Target	attributes starting with target indicates values to be returned from Data Service. It can return record or error.
Mode	Indicates Data Service should be used in View or Edit mode
RecordUpdated	edit mode does not updates record by default, to update target* attributes, use this method handler



How does Lightning Data Service Work?



Lightning Data Service

- Caches results on the client
- Invalidates cache entry when data and meta data changes
- Optimizes server calls

UI API

- Public Salesforce API to build UI
- Gives data and metadata in a single response
- UI responses respect CRUD, FLS and Sharing settings



Base Form Components

let you create custom forms in a declarative, quick and simple way without having to write Apex

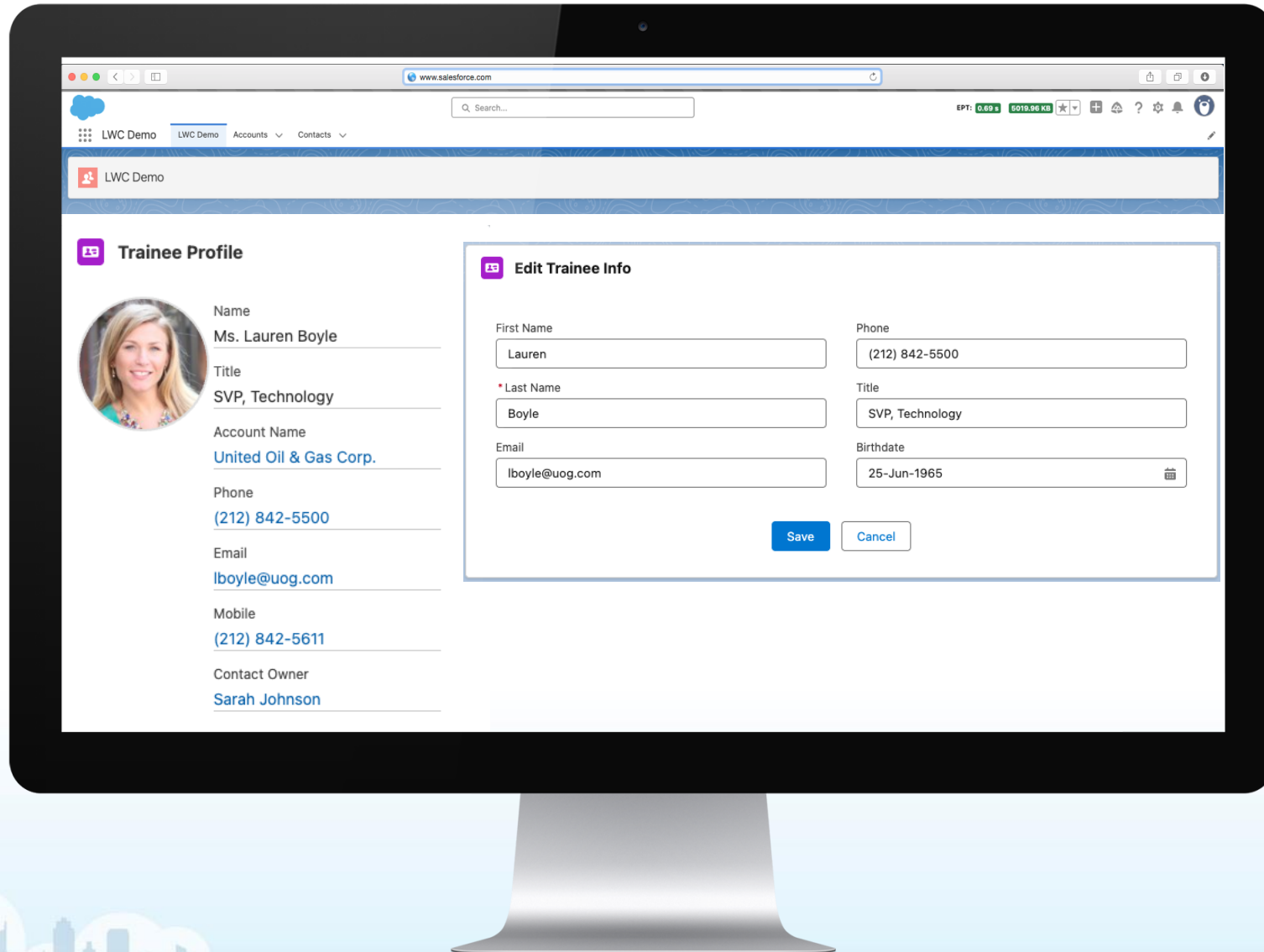
FEATURE	LIGHTNING-RECORD-FORM	LIGHTNING-RECORD-VIEW-FORM	LIGHTNING-RECORD-EDIT-FORM
Create Records	✓		✓
Edit Records	✓		✓
View Records	✓	✓	
Read-Only Mode	✓	✓	
Layout Types	✓		
Multi Column Layout	✓		
Custom Layout for Fields		✓	✓
Custom Rendering of Record Data		✓	✓

Demo

Display and Edit Trainee Info

salesforce

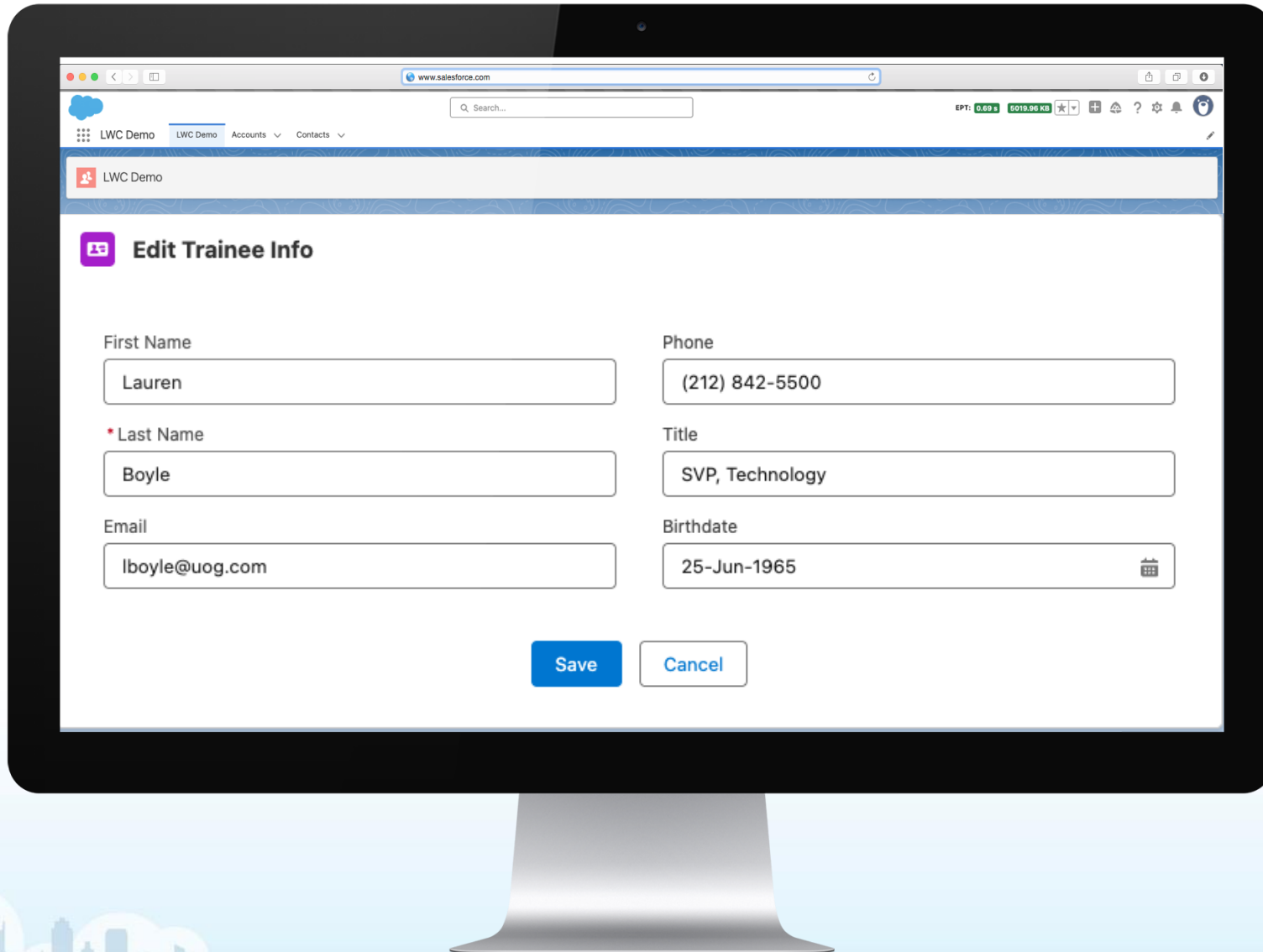
Display Trainee Details



Components to Refer

- ◆ LWC JavaScript: traineeCard.js
- ◆ LWC HTML: traineeCard.html
- ◆ LWC Meta XML: traineeCard.js-meta.xml
- ◆ LWC CSS : traineeCard.css

Edit Trainee Info



The screenshot shows a web browser window displaying the Salesforce 'Edit Trainee Info' form. The browser's address bar shows 'www.salesforce.com'. The Salesforce navigation bar includes 'LWC Demo', 'Accounts', and 'Contacts'. The form title is 'Edit Trainee Info' with a person icon. The form contains the following fields:

Field Label	Value
First Name	Lauren
Phone	(212) 842-5500
* Last Name	Boyle
Title	SVP, Technology
Email	lboyle@uog.com
Birthdate	25-Jun-1965

At the bottom of the form are two buttons: 'Save' (blue) and 'Cancel' (white with blue border).

Components to Refer

- ◆ LWC JavaScript: traineeEditCard.js
- ◆ LWC HTML: traineeEditCard.html
- ◆ LWC Meta XML: traineeEditCard.js-meta.xml



Using Wire Service to Call Apex





How the Wire Service Works

retrieve data reactively from Salesforce — including **Apex methods**, **LDS (Lightning Data Service)**, and **other adapters** — without manually handling the lifecycle or data flow.

The wire service (`@wire`) is a reactive mechanism in LWC that:

- Automatically fetches data when the component loads or when reactive parameters change.
- Refreshes data when needed (e.g., using `refreshApex()`).
- Doesn't require explicit invocation like calling a method.
- It is declarative, reactive, and built into the LWC engine.

The `@wire` delegates control flow to the Lightning Web Components engine. Delegating control is great for read operations, but it isn't great for create, update, and delete operations.

Call Apex method using wire service

salesforce

Syntax

```
import apexMethod from '@salesforce/apex/Namespace.Classname.apexMethod';  
  
@wire(apexMethod, { apexMethodParams }) propertyOrFunction;
```

Usage

```
import { LightningElement, track, wire } from 'lwc';  
import findContacts from '@salesforce/apex/ContactController.findContacts';  
  
export default class ApexWireMethodToProperty extends LightningElement { @track  
  searchKey = '';  
  @wire(findContacts, { searchKey: '$searchKey' }) contacts;  
}
```



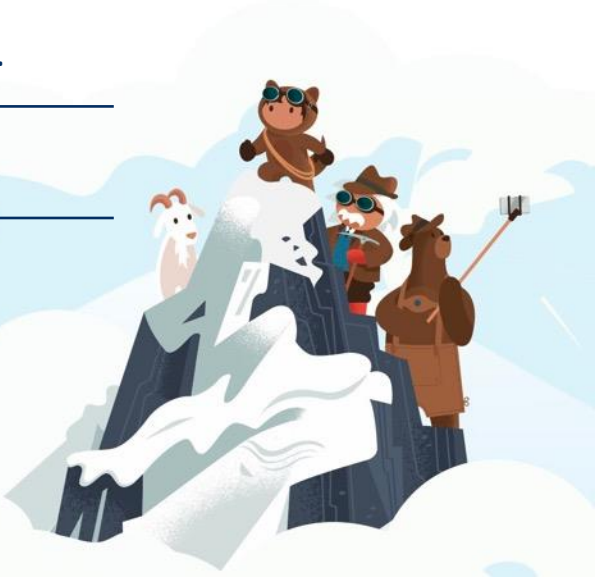


Key Modules within @salesforce

@salesforce - namespace that provides access to Salesforce-specific modules, APIs, and base components.

These modules enable developers to interact with the Salesforce platform, access data, and leverage Salesforce's UI elements within their LWC development.

@salesforce/apex	Used to interact with Apex methods (server-side code).
@salesforce/schema	Provides access to Salesforce object and field definitions.
@salesforce/resourceUrl	Allows referencing static resources like images or CSS files.
@salesforce/community	Provides access to community-related information and functionalities.
@salesforce/user	Provides access to user-related information.
@salesforce/label	Provides access to custom labels defined in Salesforce.

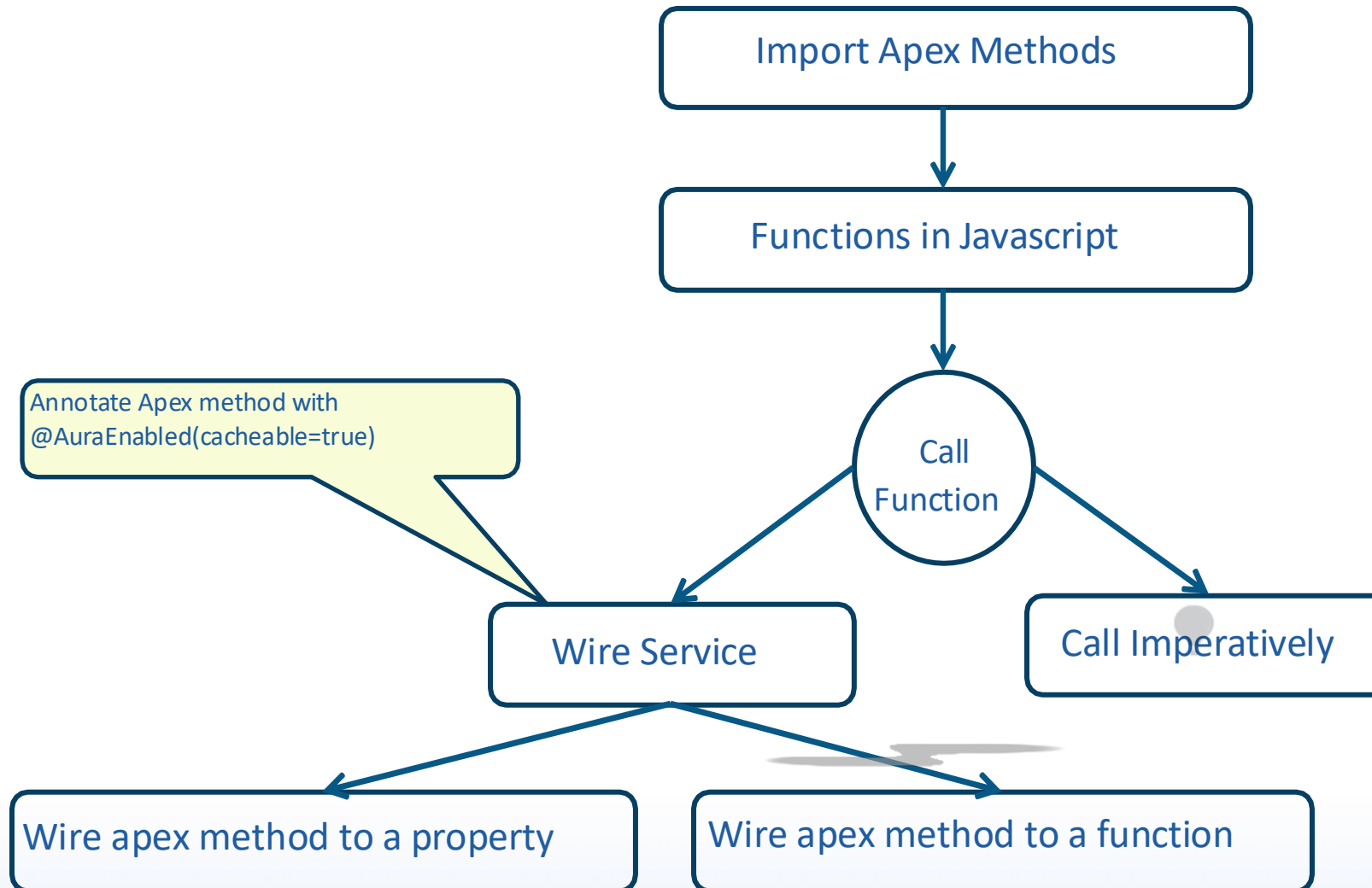




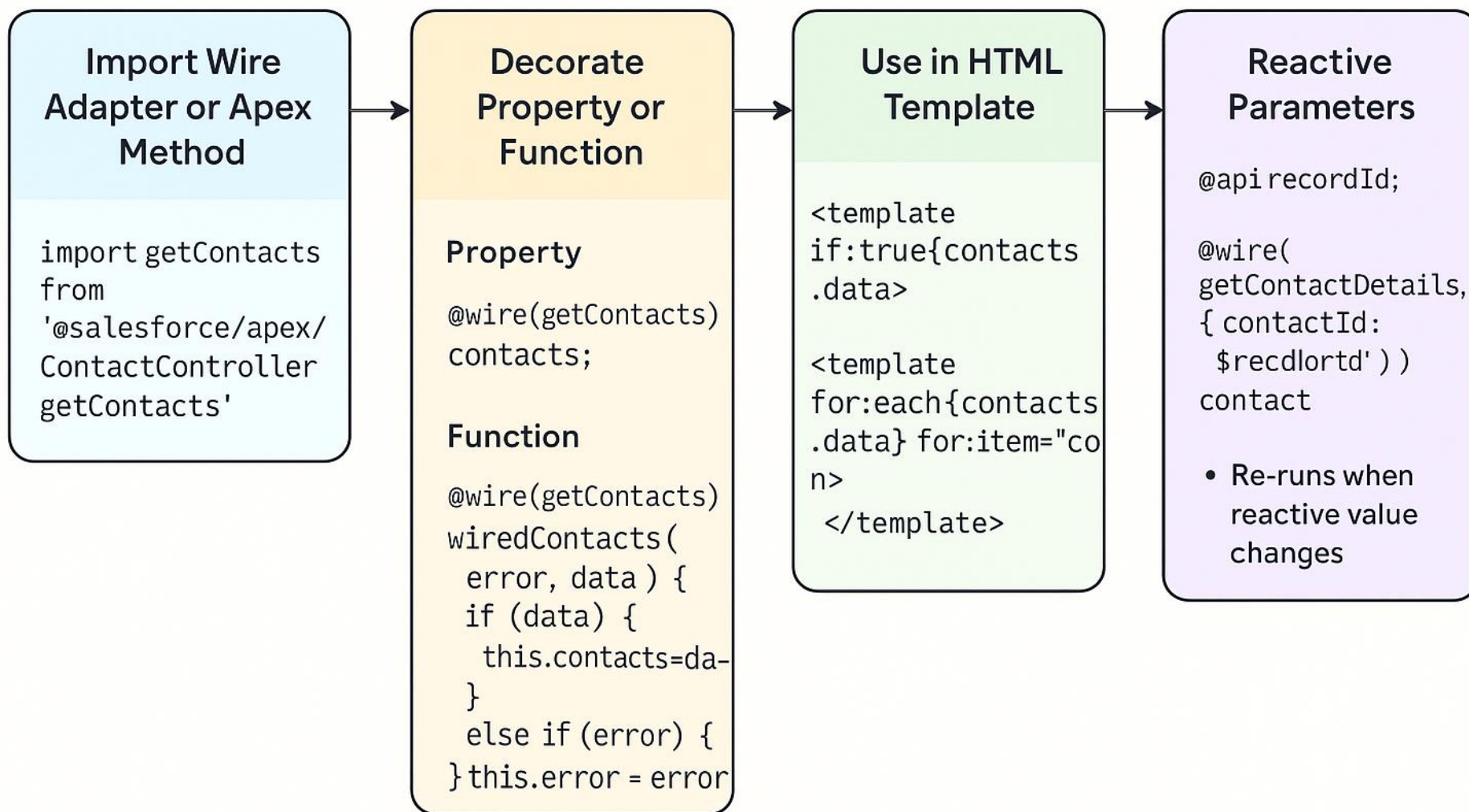
How Wire Service Works?



Call Apex Methods from Lightning Web Components



How the Wire Service works in LWC?



Get data with Wire Service

The **@wire** service in **Lightning Web Components (LWC)** also allows to pass the parameters to the Apex function being called.

Syntax

```
import { adapterId } from 'adapterModule';  
@wire(adapterId, adapterConfig) propertyOrFunction;
```

HTML

```
<template>  
  {record}  
</template>
```

```
import { LightningElement, api, wire } from 'lwc';  
import { getRecord } from 'lightning/uiRecordApi';  
  
export default class Record extends LightningElement {  
  @api recordId;  
  
  @wire(getRecord, { recordId: '$recordId', fields: ['Account.Name'] }) record;  
}
```



QUICK
TIPS

lightning/ui*Api Wire Adapters and Functions

Built on top of Lightning Data Service (LDS) and User Interface API.

Use these wire adapters and functions to work with Salesforce data and metadata.

1. Import the Wire Adapter or Apex Method

```
import { wire } from 'lwc';
import getContacts from
 '@salesforce/apex/ContactController.getContacts';
```

ContactController is the apex class where we have defined the getContacts() method to retrieve the contacts

2. Decorate Property of Function

You can wire:

- A **property**: good for using the result directly in the template.
- A **function**: gives you access to data and error.

✓ Wire to Property (auto assignment)

```
@wire(getContacts)
contacts;
```

Use contacts.data and contacts.error in the template or logic.

Control actions on data or error.

✓ Wire to Function (fine control)

```
@wire(getContacts)
wiredContacts({ error, data }) {
  if (data) {
    this.contacts = data;
  } else if (error) {
    this.error = error;
  }
}
```

3. Use in HTML Template

```
<template if:true={contacts.data}>
  <template for:each={contacts.data} for:item="con">
    <p key={con.Id}>{con.Name}</p>
  </template>
</template>
```

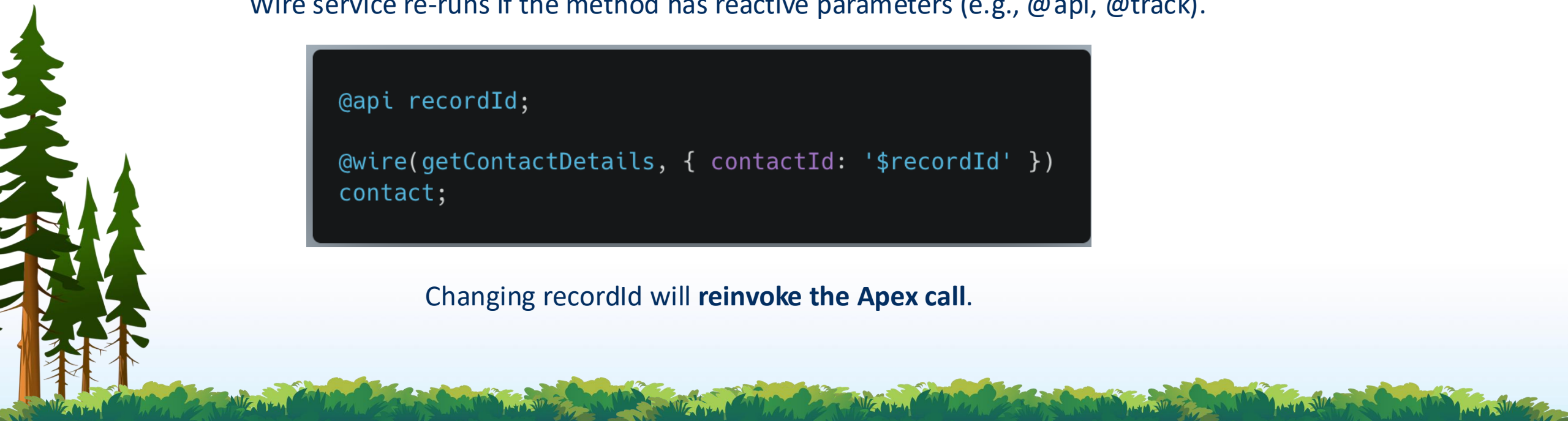
Reactive Parameters

Wire service re-runs if the method has reactive parameters (e.g., @api, @track).

```
@api recordId;

@wire(getContactDetails, { contactId: '$recordId' })
contact;
```

Changing recordId will **reinvoke the Apex call**.



Refreshing the Wire (refreshApex)



If data is updated behind the scenes (like a record edit), use `refreshApex` to re-fetch:

```
import { refreshApex } from '@salesforce/apex';

let wiredResult;

@wire(getContacts)
wiredContacts(result) {
  wiredResult = result;
  if (result.data) {
    this.contacts = result.data;
  }
}

handleRefresh() {
  refreshApex(wiredResult);
}
```



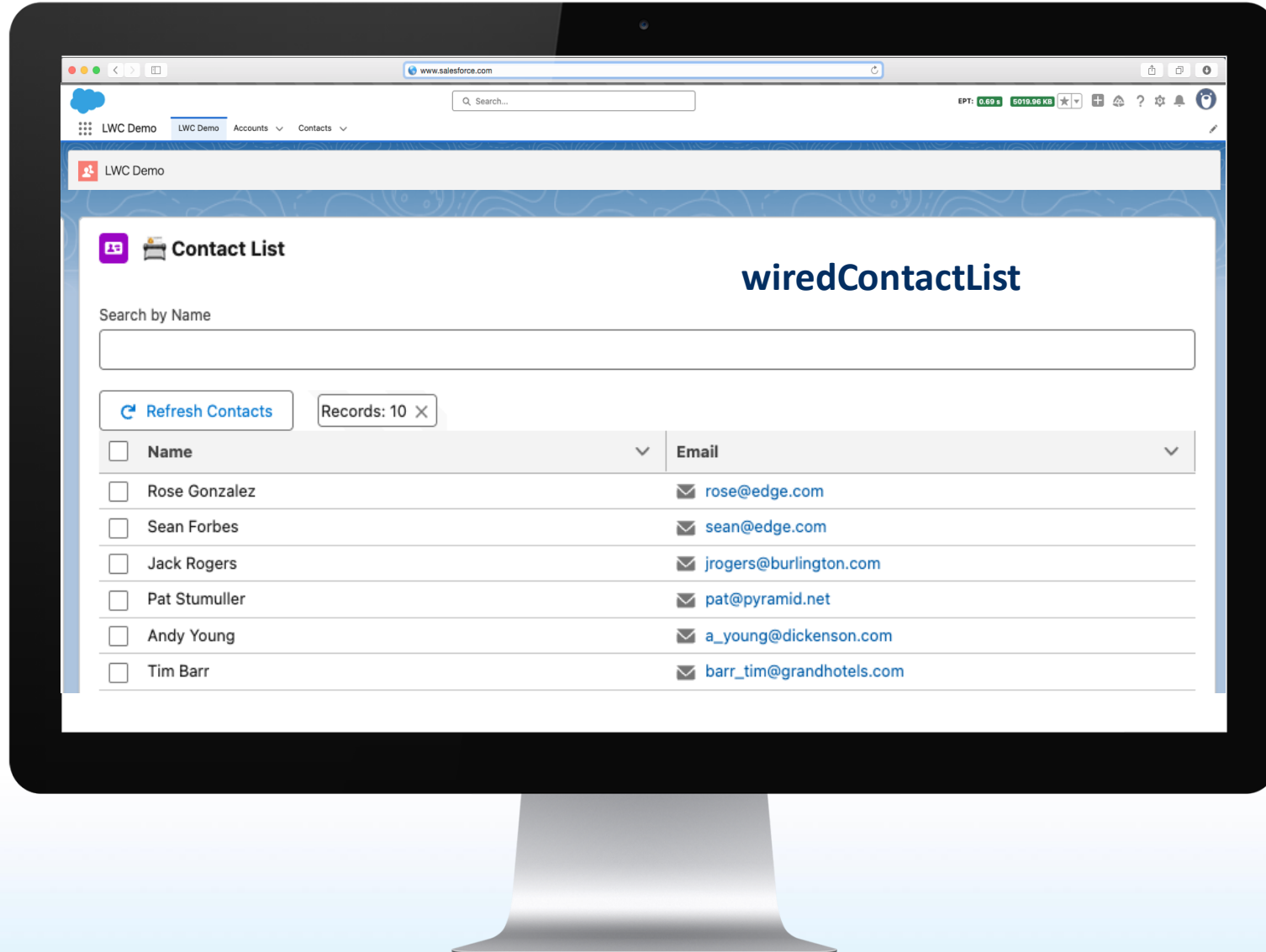
Demo

Create Trainee Card

salesforce

Lightning Web Component that uses the @wire service to fetch a list of Contacts from an Apex class.

salesforce



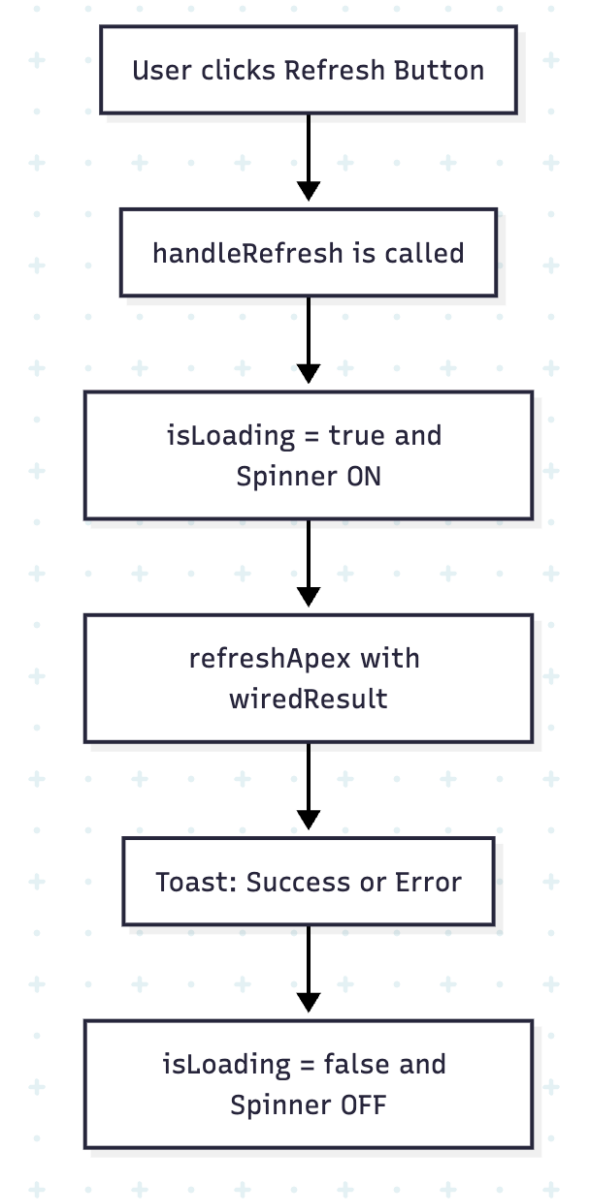
Components to Refer

- ◆ Apex Class: ContactHandler.cls
- ◆ LWC JavaScript: wiredContactList.js
- ◆ LWC HTML: wiredContactList.html
- ◆ LWC Meta XML: wiredContactList.js-meta.xml

Execution Flow of “Refresh Contacts” button

The flow describes how the @wire would help refresh the contacts list we just created.

Use the mermaid chart to represent how the flow would work. Then build the functionality. It's easier to understand for anyone and helps build the documentation while development.





Calling Apex using Imperative Method



Calling Apex Imperatively in LWC

- using a direct method call via JavaScript
- used when you want full control over when the method is invoked
- Imperative Apex call means we import the Apex method and call it manually inside our JavaScript logic.
- Useful when you need to:
 - Pass dynamic parameters.
 - Handle multiple methods conditionally.
 - Chain methods or handle complex UI flow.

```
import getContacts from '@salesforce/apex/ContactController.getContacts';

handleLoad() {
  getContacts({ accountId: this.recordId })
    .then(result => {
      this.contacts = result;
    })
    .catch(error => {
      this.error = error;
    });
}
```

Imperative methods



```
import { LightningElement, track } from 'lwc';  
import getContactList from '@salesforce/apex/ContactController.getContactList';
```

```
export default class ApexImperativeMethod extends LightningElement {
```

```
  @track contacts;
```

```
  @track error;
```

```
  handleLoad() {
```

```
    getContactList()
```

```
      .then(result => {  
        this.contacts = result;
```

```
      })
```

```
      .catch(error => {  
        this.error = error;
```

```
      });
```

```
  }
```

```
}
```

Javascript
Promise



Javascript Promises

- a Promise is an object that represents the eventual completion or failure of an asynchronous operation and its resulting value. It acts as a placeholder for a value that may not be available immediately but will be at some point in the future.
- **Key Concepts of Promises:**
- **States:** A Promise can be in one of three states:
 - **Pending:** The initial state; the asynchronous operation is still in progress.
 - **Fulfilled (or Resolved):** The operation completed successfully, and the Promise now has a resulting value.
 - **Rejected:** The operation failed, and the Promise has an error reason.
- **Immutability:** Once a Promise is settled (either fulfilled or rejected), its state and result cannot change.
- **Handlers:** Promises provide methods to attach callbacks that will execute when the Promise settles:
 - **.then():** Used to handle the success (fulfillment) or failure (rejection) of a Promise. It can take two optional arguments: a callback for fulfillment and a callback for rejection.
 - **.catch():** A shorthand for .then(null, rejectionHandler), specifically for handling rejections (errors).
 - **.finally():** Executes a callback regardless of whether the Promise was fulfilled or rejected, useful for cleanup operations.

Benefits of Promises

Improved Readability

Makes asynchronous code easier to understand and manage compared to traditional callback-based approaches (callback hell).

Chaining

Allows for sequential execution of asynchronous tasks using chaining.

Error Handling

Provides a structured way to handle errors in asynchronous operations.

Composition

Enables combining multiple Promises to manage complex asynchronous workflows (e.g., ,).

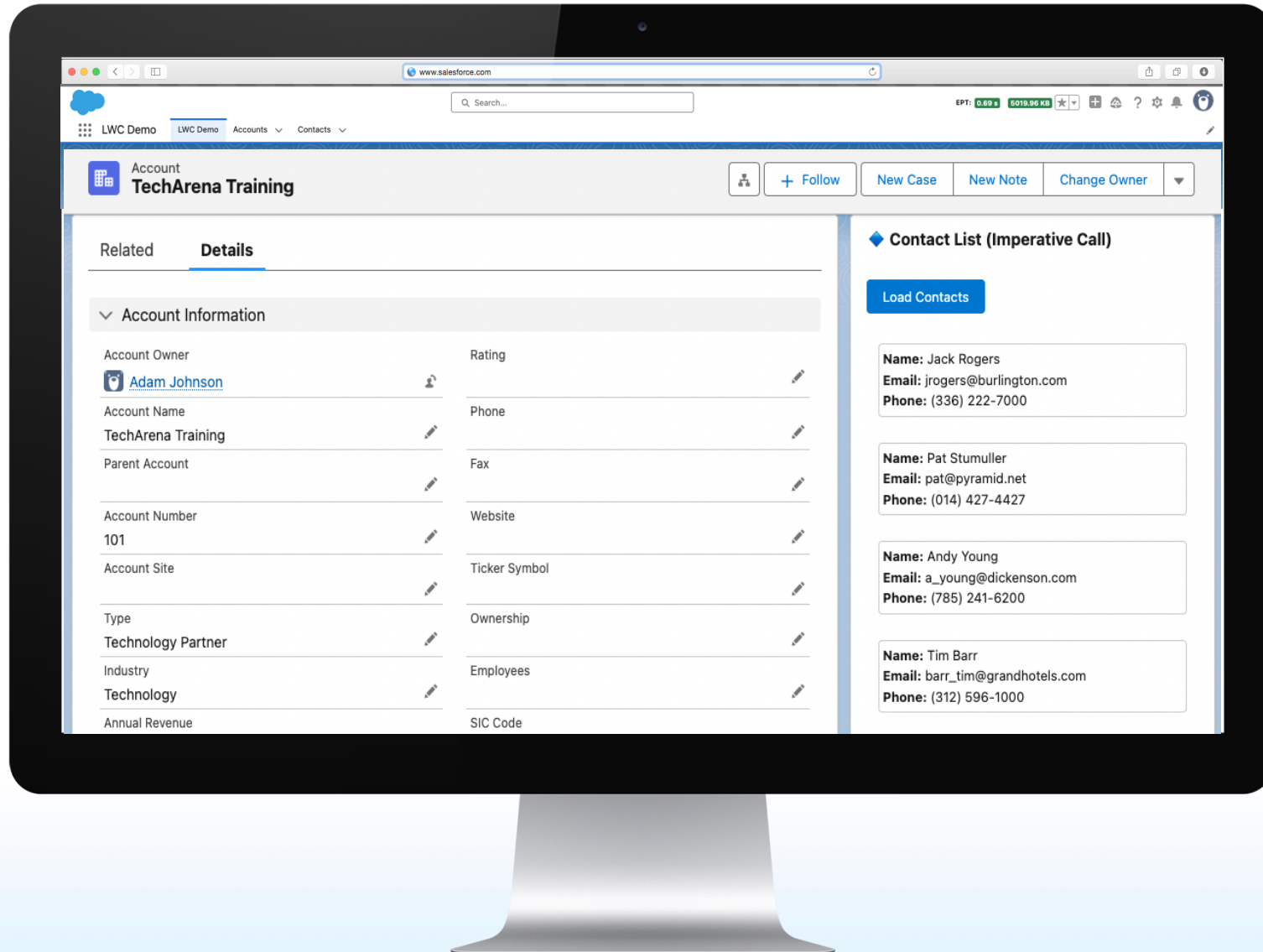


Demo

Load Contacts for Accounts

salesforce

When the user clicks 'Load Contacts' button on an Account record page, the component should fetch related contacts from Apex using the Account's Id. The fetch happens only when the user clicks, not automatically.



Components to Refer

- ◆ Apex Class: ContactHandler.cls
- ◆ LWC JavaScript: imperativeContacts.js
- ◆ LWC HTML: imperativeContacts.html
- ◆ LWC Meta XML: imperativeContacts.js-meta.xml

Javascript Breakdown

Line(s)	What It Does	Why It Matters
@api recordId;	Receives the Account Id from the record page	Allows component to know which account's contacts to fetch
@track contacts = [];	Reactive variable to store contact list	Enables real-time re-rendering of UI when contacts are updated
isLoading = false;	Controls spinner visibility	Enhances UX by showing activity feedback
handleLoadContacts()	Method triggered when button is clicked	Entry point to start Apex call
getContacts({ accountId: this.recordId })	Calls Apex method with accountId	Imperative call allows conditional/manual execution
.then(result => { ... })	Handles success: stores result and clears error	Ensures correct UI update and feedback
.catch(error => { ... })	Catches and stores errors if Apex fails	Prevents crash and shows user-friendly error
.finally(() => { ... })	Always executed; hides spinner	Ensures cleanup regardless of success or failure
showToast(...)	Shows user messages	Improves UX with contextual feedback

salesforce

References

- [LDS – Jitendra Zha](#)
- [Playground to Test](#)
- [LWC Dev Guide](#)



Rhea Sy

Lead Developer, 21st Century Fox

Nithin Angri

Program Manager, AWS



thank you

Family Reunion

